



Hands-On Assignment: CI/CD + STO using Harness (Free Tier)

Objective

Build an end-to-end CI/CD pipeline with integrated security testing (STO) using Harness and a local Kubernetes cluster.

This assignment is designed to evaluate your practical understanding of:

- CI/CD pipeline design
 - Containerization & Kubernetes
 - Shift-left security (SAST / container scanning)
 - Deployment strategies and gating
-

Requirements

Tools (All free-tier compatible)

- Harness Free Tier account
 - GitHub repository
 - Docker
 - Local Kubernetes cluster ([minikube](#) or [kind](#))
 - Or use any of the cloud free environments for infrastructure
-

Application Setup

Create or use a simple application:

- Node.js / Python / Java (any is fine)

- Must expose a basic HTTP endpoint (`/health` or `/`)

Example:

- Returns “Hello World” or simple JSON response
-

Pipeline Requirements

1. CI Stage (Build & Push)

- Clone application repo
 - Build Docker image
 - Tag image with commit SHA or build number
 - Push image to Docker Hub (or any public registry)
-

2. STO Stage (Security Testing)

Integrate at least **one** of the following:

Required:

- Container scan

Optional (bonus):

- SAST scan
-

Security Gate (Mandatory)

- Fail the pipeline if:
 - Any **CRITICAL vulnerabilities** are found
-

3. CD Stage (Deploy to Kubernetes)

- Deploy the application to local Kubernetes
- Use:
 - Kubernetes manifest (YAML) OR

- Harness Kubernetes deployment
-

Deployment Requirements

- Use rolling deployment strategy
 - Application should be accessible via:
 - NodePort / port-forward / ingress
-

4. Post-Deployment Validation

- Add a step to:
 - Call application endpoint (`/health`)
 - Verify successful response
-

★ Bonus (Any one)

You can earn extra points by implementing:

Shift-Left Security

- Trigger pipeline on PR
 - Run scans before merge
-

Artifact Promotion

- Build image once
 - Promote across environments (dev → staging → prod)
-

GitOps Flow

- Maintain Kubernetes manifests in a separate repo
 - Update manifests via PR
-

Pipeline Enhancements

- Parameterized pipelines
 - Reusable templates
 - Secrets management (no hardcoding credentials)
-



Failure Handling

- Add rollback if deployment fails
 - Add retry / backoff logic
-



Submission Requirements

Please provide:

1. **GitHub Repository**
 - Application code
 - Kubernetes manifests
 - Any scripts used
 2. **Harness Pipeline**
 - Exported YAML and Execution Link
 3. **README.md (Important)**

Include:

 - Setup instructions
 - Architecture / pipeline explanation
 - How security gating works
 - Any assumptions made
 4. **Demo**
 - Live walkthrough (Round 3)
-



Notes

- You may simplify where needed, but explain trade-offs
 - Focus on demonstrating understanding, not just making it “work”
 - If something doesn’t work, document your approach and blockers
-

 **Good Luck!**

We're more interested in how you think and design systems than perfection. Keep it simple, but intentional.